

BE10X · LEARN WEB DEVELOPMENT WITH AI

Part 2 — Trainer & Student Handbook

The Mega Prompt warm-up · Supabase · Admin Dashboard · Git & GitHub · GitHub Pages

MEGA PROMPT

PROMPTS LIBRARY

TRAINING GUIDE

Combined edition · color-coded · syntax-highlighted · ready to share

The Mega Prompt — A Visual Glow-Up Before Part 2

- ✅ Before you paste — quick checklist
- 🧠 The five prompting rules (still apply)
- 📄 THE MEGA PROMPT — copy everything inside the box
- 🎨 What this prompt changes (cheat-sheet for the room)
- ✅ Verify it works (do this together in class — 3 minutes)
- 🕒 Now you're ready for Part 2

Part 2 — Prompt Library (Student Copy-Paste Edition)

- ⚙️ Setup (do once before Prompt 1)
-  Pre-flight (recommended) — Run the **Mega Prompt** first
- The Five Rules (still apply — keep them visible)

Prompt 1 — Generate the database table (SQL)

(Optional) Prompt 1b — Lock the table down with Row-Level Security

Prompt 2 — Wire the contact form to Supabase

Prompt 3 — The Debug Template (use this every single time something breaks)

Prompt 4 — Build the Admin Dashboard

Prompt 5 — Protect your secrets BEFORE the first `git push`

- 🔧 Push the project to GitHub (terminal commands — not a prompt)
- 🚀 Deploy live with GitHub Pages (six clicks)

Bonus — Rapid Prototyping: Timer App (10 minutes, deployed)

- Prompt 6 — Timer v1 (build it)
- Prompt 7 — Timer v2 (iterate without breaking)
- 🎓 You did it

Part 2 — Training Guide (Trainer Edition)

- Learn Web Development with AI — From Static Site to Deployed Full-Stack App
- Audience reminder (read before you teach)
- The Bridge from Part 1
- How This Guide Is Structured

Module 0 — The Mega Prompt (warm-up · ~15 minutes)

- Module 1 — Full-Stack in One Sentence
- Module 2 — Databases for Humans
- 🟢 Q&A Break #1 (5 min)
- Module 3 — Generating SQL with AI
- Module 4 — Connect Front-End to the Database
- Module 5 — AI-Assisted Debugging (Live)
- 🟢 Q&A Break #2 (10 min)
- Module 6 — The Admin Dashboard
- 🟢 Q&A Break #3 (5 min)
- Module 7 — Version Control with Git & GitHub
- Module 8 — Deploy with GitHub Pages
- 🟢 Q&A Break #4 (10 min)

[Module 9 — Rapid Prototyping Bonus: A Timer App](#)

[Module 10 — Wrap-up](#)

 [Q&A Break #5 — Final \(15 min\)](#)

[Appendix — The Five Rules, restated \(still true\)](#)



The Mega Prompt — A Visual Glow-Up Before Part 2

When to run this: at the very start of Part 2, **before** Prompt 1. One copy-paste, one Cursor run, ~2–4 minutes of generation time.

What it does: takes the 4-page portfolio you built in Part 1 and turns it into a *visually stunning, animated, polished* product — so when we add a database in Part 2, the experience already feels premium.

Why it exists: Part 1 proved you can build. The Mega Prompt proves AI + HTML + CSS + JS can be *beautiful*. Same five files. No new libraries. Runs locally with a simple file-open. The "wow" sets the tone for the rest of the day.



Before you paste — quick checklist

1. The folder `aanya-portfolio` from Part 1 is open in Cursor.
2. You can see all 6 files in the sidebar: `index.html`, `about.html`, `projects.html`, `contact.html`, `styles.css`, `script.js`.
3. Cursor Chat is open (`Cmd + L` / `Ctrl + L`) and **Agent** mode is selected (top of the chat panel).
4. You have a 2-minute coffee — Agent will edit multiple files and you should let it finish.



The five prompting rules (still apply)

1. **Numbers beat adjectives.** Every visual choice below has a hex code, pixel value, or duration.
2. **Persona first.** The prompt tells the AI exactly who it is and who you are.
3. **Consolidate.** This is one prompt that does the work of fifteen.
4. **Iterate with anchor + delta.** We explicitly tell the AI what to keep and what to change.
5. **Demand the "why".** We ask for a short summary at the end so you learn from the changes.



THE MEGA PROMPT — copy everything inside the box

Tip: click into the code block, press `Cmd/Ctrl + A` to select all, then `Cmd/Ctrl + C` to copy. Paste into Cursor Chat (Agent mode) and press Enter. Do **not** edit the prompt before sending — the specifics are what make it work the same way for everyone in the room.

You are a senior front-end designer-developer pair-programming with a complete beginner. We are upgrading an EXISTING 4-page static portfolio called "Aanya Sharma – Senior UX Designer".

THE EXISTING FILES (do not rename, do not delete, do not move):

- index.html (Home: hero + skills grid + footer)
- about.html (About: header + 2-col bio + timeline + footer)
- projects.html (Projects: header + 6 project cards + footer)
- contact.html (Contact: header + 2-col with contact card + form + footer)
- styles.css (single shared stylesheet using CSS variables in :root)
- script.js (vanilla JS: contact form handler + hamburger toggle)

GOAL

Turn this into a visually stunning, modern, animated portfolio while keeping 100% of the existing content (every heading, paragraph, link, skill, project, timeline entry, and form field). Only LAYOUT, STYLE, and MICRO-INTERACTIONS should change. All four pages must continue to work when opened directly by double-clicking the HTML file (no build tools, no npm, no frameworks).

GLOBAL DESIGN SYSTEM – overwrite the existing :root variables in styles.css with this exact token set, and refactor the stylesheet to use these tokens everywhere a colour, radius, shadow or duration appears:

```
/* Light theme (default) */
--bg: #f7f7fb;
--surface: #ffffff;
--surface-2: #f1f1f8;
--text: #0b0b14;
--text-muted: #5b5b78;
--border: rgba(11,11,20,.08);
--primary: #6d5efc; /* indigo-violet */
--primary-2: #b06dfc; /* magenta-violet */
--accent: #22d3ee; /* cyan */
--success: #16a34a;
--danger: #ef4444;
--shadow-sm: 0 2px 6px rgba(11,11,20,.06);
--shadow-md: 0 12px 32px -8px rgba(109,94,252,.18);
--shadow-lg: 0 30px 60px -20px rgba(109,94,252,.35);
--radius-sm: 10px;
--radius-md: 16px;
--radius-lg: 24px;
--radius-pill: 999px;
--ease: cubic-bezier(.2,.7,.2,1);
--dur-fast: 180ms;
--dur: 320ms;
--dur-slow: 650ms;
--grad-hero: linear-gradient(135deg,#6d5efc 0%,#b06dfc 50%,#22d3ee 100%);
--grad-text: linear-gradient(135deg,#0b0b14 0%,#6d5efc 60%,#22d3ee 100%);
```

Also add a [data-theme="dark"] override block on :root with:

```
--bg:#08090f; --surface:#13141f; --surface-2:#1c1d2e;
--text:#f5f5fa; --text-muted:#9ca3b7;
--border:rgba(245,245,250,.10);
--shadow-sm:0 2px 8px rgba(0,0,0,.4);
```

```
--shadow-md:0 14px 40px -10px rgba(34,211,238,.20);
--shadow-lg:0 30px 80px -20px rgba(109,94,252,.45);
--grad-text: linear-gradient(135deg,#ffffff 0%,#b06dfc 60%,#22d3ee 100%);
```

Add smooth theme transitions: body, *, *::before, *::after should transition background-color, color, border-color, box-shadow over var(--dur) var(--ease).

NAVIGATION (all 4 pages – keep the existing structure, restyle only)

- Glassmorphism: background rgba(255,255,255,.65) in light / rgba(19,20,31,.55) in dark, backdrop-filter: blur(18px) saturate(140%), 1px bottom border using var(--border).
 - Add a class "is-scrolled" to the <nav> when window.scrolly > 8: shrinks height from 72px → 56px, deepens the shadow to var(--shadow-md), and slightly increases background opacity. Use a passive scroll listener in script.js (append only – do not remove the existing hamburger code).
 - Brand text "Aanya Sharma" must use background:var(--grad-hero); -webkit-background-clip:text; color:transparent; font-weight:700.
 - Add a circular theme-toggle button to the right of the nav links (sun ☀️ when in dark mode, moon 🌙 when in light mode). It must:
 - * Be 36×36px, var(--radius-pill), var(--surface-2) background, 1px border.
 - * On click, toggle html[data-theme] between "light" and "dark".
 - * Persist the choice in localStorage under key "aanya-theme".
 - * On page load, read localStorage; if missing, fall back to window.matchMedia('(prefers-color-scheme: dark)').
- Add this script to the TOP of script.js (so theme is set before paint) and add the toggle button HTML to the <nav> of all 4 pages.
- The active nav link gets a 2px-thick underline using var(--grad-hero) that animates from 0% → 100% width over var(--dur).

HERO (index.html only – keep all existing copy)

- Replace the flat gradient background with a layered animated background:
 1. Base: var(--surface).
 2. Two large soft "blobs" using ::before and ::after on .hero, each 60vw wide, filter: blur(80px), opacity .55, mix-blend-mode: screen in dark / multiply in light. Colours: #6d5efc and #22d3ee. Animate them with a 12s ease-in-out infinite keyframe that gently translates them ±40px in x and y (different phases) – call the keyframe "blobFloat".
 3. A subtle SVG noise overlay (data-URI, 120×120 fractal noise, opacity .04) to add texture.
- Headline ("I design products people actually enjoy using.") uses background: var(--grad-text); -webkit-background-clip:text; color:transparent; font-size clamp(40px, 6vw, 84px); line-height 1.05.
- Add a small animated "scroll" indicator at the bottom-centre of the hero: a 22×36px rounded rectangle with a 4px dot inside that travels top→bottom every 1.6s.
- Primary CTA button: var(--grad-hero) background, white text, var(--radius-pill), 18px/36px padding, var(--shadow-md). On hover: translateY(-3px), shadow becomes var(--shadow-lg), and a soft white "shine" sweeps from left to right using a pseudo-element with linear-gradient(120deg, transparent, rgba(255,255,255,.4), transparent) animated over 700ms.

SKILLS GRID (index.html – keep all 6 cards and their copy)

- Card background: var(--surface), 1px solid var(--border), var(--radius-lg), 28px padding, var(--shadow-sm).
- On hover: translateY(-8px), box-shadow var(--shadow-lg), and the border

- becomes a 1px transparent border with a gradient ring using `background-clip: padding-box, border-image: var(--grad-hero) 1`.
- Each icon sits inside a 56x56px `var(--surface-2)` rounded square (`radius-md`), `font-size 28px`, with a soft glow ring (`box-shadow: 0 0 0 6px rgba(109,94,252,.08)`).
- Add a faint "shine-on-hover" diagonal sweep across each card using `::after` with `transform: translateX(-100%) → translateX(100%)` on hover, `var(--dur-slow)`, `var(--ease)`.

ABOUT PAGE

- Page header: keep the existing h1 and subtitle. Add an animated SVG "blob" background behind the h1 (same colours as hero, much smaller, opacity .25).
- Two-column section: portrait gets a 8px gradient border using `border-image: var(--grad-hero) 1, radius-lg, var(--shadow-lg)`, and a subtle tilt of -3deg on desktop only (>900px). On hover, tilt smoothly returns to 0.
- Add a "stats strip" ABOVE the two-column section with three counters:
 - "5+ years experience" · "30+ shipped projects" · "12 awards"
 Each number must animate from 0 to its final value over 1.4s when the strip scrolls into view (use `IntersectionObserver`). Use existing-content tone of voice; numbers above are placeholders consistent with the bio.
- Timeline: keep the 4 entries. Restyle to a vertical "rail" using a `var(--grad-hero) 2px` line; each year dot is a 14px circle filled with `var(--primary)` and a 6px outer glow. Each row fades+slides in from the left when it enters the viewport (`IntersectionObserver`, `.reveal` class, see GLOBAL ANIMATION).

PROJECTS PAGE

- Replace placeholder.co URLs only if needed for new background colours that match the system. Keep the existing 6 titles, descriptions, tags, and the bg colours 4f46e5, 0ea5e9, 10b981, f59e0b, ef4444, 8b5cf6.
- Card upgrade:
 - * Image area becomes a 16:9 wrapper with `overflow:hidden + radius-md`.
 - * On card hover, the image scales to 1.06 over `var(--dur)`.
 - * A `linear-gradient(180deg, transparent 55%, rgba(0,0,0,.55) 100%)` overlay appears on hover, and the "View Case Study →" link slides up from `translateY(8px) → 0` with opacity 0 → 1.
 - * Card itself uses `var(--surface)`, `radius-lg`, 1px `var(--border)`, tilts very slightly on hover: `rotate(-.4deg) translateY(-6px)`, shadow `var(--shadow-lg)`.
- Tags: pill style stays, but on hover they swap to `var(--grad-hero)` background with white text over `var(--dur-fast)`.
- Add a thin horizontal "tag marquee" ABOVE the grid: a single row of small pills ("Fintech · Edtech · Healthcare · B2B · AI/ML · Design Systems · Marketplace · Data Viz · Internal Tools · UX Writing") that scrolls right-left infinitely over 30s using a CSS `@keyframes` (no JS). Duplicate the row twice inside a wrapper with `overflow:hidden` so the loop appears seamless. Pause on hover.

CONTACT PAGE

- Left contact card upgrades to a "frosted" card: `var(--surface)`, `radius-lg`, 1px `var(--border)`, `var(--shadow-md)`. Replace the social links list with three large icon-buttons (LinkedIn 🏠, Dribbble 🎨, Twitter 🐦) – each a 56x56 rounded square with `var(--surface-2)` bg, hover scales to 1.08 and background fills with `var(--grad-hero)` (white emoji stays).
- Form inputs:
 - * Float-label style: label sits inside the input at 50% Y; on focus or when the input has a value, the label scales to .82 and moves to the top-left

corner with a `var(--bg)` background chip. Implement using CSS only via `:placeholder-shown` and `:not(:placeholder-shown)` – give each input a `placeholder=" "` (single space) so the trick works.

- * Inputs: `var(--surface)`, 1.5px `var(--border)`, `radius-md`, 16px padding, transition `border-color` and `box-shadow`. On focus: `border var(--primary)`, `box-shadow 0 0 0 4px rgba(109,94,252,.18)`.
- Submit button: `var(--grad-hero)`, white, `var(--radius-pill)`, 14px/28px padding, `var(--shadow-md)`. On hover: same "shine sweep" as the hero CTA.
- DO NOT change the existing JS submit handler logic – only restyle. Keep the `success-message` element and id intact so Part 2 can wire it to the database.

FOOTER (all 4 pages)

- Background `var(--surface-2)`, top border 1px `var(--border)`.
- Add a single horizontal accent line above the columns using `var(--grad-hero)`, height 2px, opacity .6.
- Restyle the social emoji list to match the contact page (compact icon buttons, same hover behaviour).
- Keep the copyright text but place a small animated heart (♥) that pulses every 1.4s next to "Built with curiosity and Cursor."

GLOBAL ANIMATIONS (append to `script.js` – do not remove existing code)

1. Reveal-on-scroll:

- Add a vanilla `IntersectionObserver` that watches every element with class `".reveal"`. When it enters the viewport, add `".is-visible"` and unobserve it.
- In `styles.css`, `".reveal"` starts with opacity 0 and transform `translateY(28px)`, and `".reveal.is-visible"` goes to opacity 1 / `translateY(0)` with `var(--dur-slow)` `var(--ease)`.
- Add the `"reveal"` class (in the HTML) to:
 - * Every skill card on `index.html`.
 - * Each timeline entry on `about.html` (with staggered `transition-delay`: 50ms, 150ms, 250ms, 350ms).
 - * Every project card on `projects.html` (staggered 50ms × index).
 - * The contact card and form on `contact.html`.

2. Top page scroll-progress bar:

- 3px tall, fixed `top:0`, full width, background `var(--grad-hero)`, width scales from 0% → 100% based on `(scrollTop / (scrollHeight - innerHeight))`.
- Inject the bar element via script on every page (so we don't have to edit each HTML manually). Use `requestAnimationFrame` to throttle.

3. Number counter for the About-page stats strip (already described above).

4. Page-load splash:

- On every page load, briefly show a centred minimal splash: black-out overlay with the gradient brand "Aanya Sharma" fading in then out over 900ms total, then removing itself from the DOM. Inject via JS so we don't touch every HTML file.

5. Custom focus-visible ring: any focused interactive element gets

`outline: 2px solid var(--primary); outline-offset: 3px; border-radius inherited`. Skip the default browser dotted ring.

TYPOGRAPHY

- Keep the existing Google Fonts import (Inter + Fraunces). Add weight 800 to Fraunces if not already requested. Use `letter-spacing: -.02em` on `h1/h2` for a modern editorial feel.

ACCESSIBILITY (non-negotiable)

- All decorative animations must respect

- ```
@media (prefers-reduced-motion: reduce) { *, ::before, ::after {
 animation-duration: .001ms !important; transition-duration: .001ms !important;
 animation-iteration-count: 1 !important; scroll-behavior: auto !important; } }
```
- Theme toggle must have aria-label that updates ("Switch to dark mode" / "Switch to light mode").
  - All colour combinations must keep contrast ratio  $\geq 4.5$  for body text. If any token combo fails, pick the closer-to-pure variant and tell me what you adjusted.

#### CODE ORGANISATION

- All new CSS goes under clearly labelled section comments inserted in this order at the end of styles.css:

```
/* === DESIGN TOKENS === */
/* === GLOBAL ANIMATIONS & UTILITIES === */
/* === NAV (GLASS) === */
/* === HERO (BLOBS) === */
/* === SKILLS (UPGRADED) === */
/* === ABOUT (STATS + TIMELINE) === */
/* === PROJECTS (MARQUEE + CARDS) === */
/* === CONTACT (FROSTED + FLOAT-LABELS) === */
/* === FOOTER (UPGRADED) === */
/* === REDUCED MOTION === */
```

Leave the existing per-section styles in place; override or extend them cleanly – do not delete prior rules unless they directly conflict.

- New JS in script.js goes under banner comments in this order:

```
// === THEME (must run first) ===
// === SCROLL PROGRESS BAR ===
// === REVEAL ON SCROLL ===
// === NAV SHRINK ON SCROLL ===
// === ABOUT STATS COUNTERS ===
// === PAGE-LOAD SPLASH ===
```

Preserve the existing hamburger toggle and contact-form handler intact.

#### DELIVERABLES – what I expect when you finish

1. All 4 HTML files updated (theme toggle in nav, reveal classes on the right elements, marquee on projects, stats strip on about, splash injected by JS so no HTML change needed for it).
2. styles.css extended with all sections above.
3. script.js extended with all behaviours above (existing code preserved).
4. A short summary at the end of your message containing:
  - The list of files you touched.
  - Three things to test in the browser (e.g., "toggle theme", "scroll the home page", "submit the contact form").
  - One sentence on the single most impactful change visually, and why.

#### CONSTRAINTS / NON-NEGOTIABLES

- No new libraries, no npm, no build step. Pure HTML/CSS/JS that runs by double-clicking the file.
- Do not change copy, headings, link targets, form field names, or element IDs.
- Do not break the Part 1 hamburger menu or the contact-form success message.
- Mobile responsiveness from Part 1 must continue to work; verify by sketching any new media-query overrides at the bottom of styles.css.

## 🎨 What this prompt changes (cheat-sheet for the room)

Use this list to follow along while Cursor edits — and to debug if something looks off.

| LAYER                                             | WHAT CHANGES                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|---------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 🟠 <b>HTML</b> (all 4 pages)                       | A theme-toggle button is added to the nav. The <code>.reveal</code> class is sprinkled on cards, timeline rows, contact panels. Projects page gets a marquee strip above the grid. About page gets a stats strip above the two-column section. No copy or link is touched.                                                                                                                                                                                         |
| 🟦 <b>CSS</b><br>( <code>styles.css</code> )       | New <code>:root</code> design tokens (colours, shadows, radii, easings, gradients) + a <code>[data-theme="dark"]</code> override. Glassmorphism nav, animated hero blobs + noise, upgraded skill cards with gradient borders & shine, frosted contact card, float-label inputs, project marquee + image-zoom hover, scroll-progress bar styles, <code>.reveal</code> keyframes, reduced-motion guard. Everything appended under clearly labelled section comments. |
| 🟡 <b>JavaScript</b><br>( <code>script.js</code> ) | Six new modules added on top of Part 1 code: (1) theme persistence via <code>localStorage</code> , (2) scroll-progress bar, (3) <code>IntersectionObserver</code> -driven reveal animations, (4) nav-shrink on scroll, (5) animated number counters on About, (6) one-time page-load splash. Existing hamburger toggle and form handler are left untouched.                                                                                                        |
| 🟢 <b>New files</b>                                | None. We re-use the same 6 files from Part 1.                                                                                                                                                                                                                                                                                                                                                                                                                      |

## ✅ Verify it works (do this together in class — 3 minutes)

1. Double-click `index.html` — you should see:

- A subtle splash with the brand name, then the home page.
- The hero gradient blobs gently float.
- Skill cards lift on hover with a glowing border.
- The nav shrinks when you scroll down.

2. Click the moon/sun button in the nav — entire site flips between **dark** and **light**. Refresh — your choice is remembered.

3. Open `about.html` — the three counters animate from 0 when you scroll to them; the timeline rows fade in one after another.

4. Open `projects.html` — tags marquee scrolls in the background, images zoom subtly on hover, "View Case Study →" slides up.

5. Open `contact.html` — labels float when you focus an input; submit a test message → green success appears (same behaviour as Part 1; we wire it to a real database in Prompt 2 of Part 2).

6. Resize the window to phone width — hamburger still works, layouts still stack.

If any of these feel off, jump to the **Debug template** in `prompts.md` (Prompt 3 of Part 2) — paste the exact error or describe the symptom and Cursor will surgically fix it.



## Now you're ready for Part 2

Open `prompts.md` and start with **Prompt 1 — Generate the database SQL**.

# Part 2 — Prompt Library (Student Copy-Paste Edition)

## How to use this file

1. Open it side-by-side with Cursor (Agent mode).
2. Run prompts **in order**, top to bottom.
3. For each prompt:
  - read the short **Goal** (one line — what we're trying to achieve),
  - copy the **Prompt** box into Cursor Chat and press Enter,
  - glance at the **What this changes in your project** card so you know what HTML / CSS / JS pieces are moving,
  - skim **Why the prompt is written this way** so you build prompt-engineering instinct, not just muscle memory,
  - run the quick **Verify** step before moving on.
4. If anything fails, jump to **Prompt 3 — The Debug Template** and follow it.

You'll still have your `aanya-portfolio` folder from Part 1. We continue right where we stopped.



## Setup (do once before Prompt 1)

1. Create a free **Supabase** account → [supabase.com](https://supabase.com) → sign in with GitHub.
2. Click **New project** → name `web-dev-with-ai` → save the DB password somewhere safe → pick the region closest to you (Mumbai / Singapore for India).
3. Create a free **GitHub** account → [github.com](https://github.com) (if you don't have one).
4. Install **Git** on your laptop:
  - **macOS:** `brew install git`
  - **Windows:** download from [git-scm.com/download/win](https://git-scm.com/download/win)
5. Once installed, run these two commands once in any terminal — they tell Git who you are:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```



## Pre-flight (recommended) — Run the Mega Prompt first

Open `mega-prompt.md` (next to this file). It contains ONE big prompt that turns your Part 1 portfolio into a visually stunning, animated, dark-mode-ready product in a single Cursor run. Do this **before** Prompt 1 below so the rest of Part 2 lands on a beautiful canvas.

When you're back, continue here.

---

## The Five Rules (still apply — keep them visible)

1. **Numbers beat adjectives.** ("16px padding", not "good spacing".)
  2. **Persona before prompt.** Tell the AI who it is and who you are.
  3. **Consolidate.** One big prompt with bullet points beats five small prompts.
  4. **Iterate with anchor + delta.** "*This part* is too big — make it 18px."
  5. **Demand the "why".** Ask the AI to explain its choices in plain English.
-

# Prompt 1 — Generate the database table (SQL)

**Goal:** create a `form` table in Supabase that will store every contact-form message your website receives.

**Prompt — copy into Cursor Chat (or ChatGPT — either works):**

You are a Postgres expert helping a complete beginner.

I am using Supabase. Please write ONE single CREATE TABLE statement for a table named "form" that will store contact-form submissions from my website.

The columns I need are:

1. `id` — bigint, auto-incrementing, primary key
2. `created_at` — timestampz, default now()
3. `full_name` — text, not null
4. `email` — text, not null
5. `subject` — text, not null
6. `message` — text, not null
7. `is_read` — boolean, default false, not null

After the SQL, add a 3-line plain-English explanation of what each column does and why we chose these data types — pretend you're explaining it to someone who has never seen SQL before.

**After Cursor / ChatGPT gives you the SQL:** In Supabase → left sidebar → **SQL Editor** → paste → **Run**. Then go to **Table Editor** and confirm the `form` table appears with all 7 columns.

**What this changes in your project:**

-  **HTML / CSS / JS:** nothing yet — we're only creating the database.
-  **Supabase (cloud):** a new table named `form` with 7 columns is created in your project.

**Why the prompt is written this way (prompt-engineering lens):**

-  **Persona** — "Postgres expert helping a complete beginner" → output is correct *and* explained simply.
-  **Numbered, exact specs** — every column has a name + type → AI cannot guess wrong.
-  **Demand the "why"** (Rule #5) → you learn while it builds.

**Verify:** in Supabase Table Editor you can see the `form` table; click "Insert row" to confirm the columns are correct, then cancel — we'll insert through our website.

## (Optional) Prompt 1b — Lock the table down with Row-Level Security

Run this only if you want production-grade security from day one. Otherwise the trainer will disable RLS for

the classroom and we'll cover hardening in homework.

For my Supabase table "form", please write the exact SQL to:

1. Enable row-level security on the table.
2. Add a policy that allows ANYONE (the "anon" role) to INSERT a new row.
3. Add a policy that allows ONLY authenticated users to SELECT rows.

Explain each policy in one sentence so a beginner can repeat it back.

**Why this prompt:** "anon insert, authenticated select" is the standard pattern for "public submits, admin reads".

Asking the AI to *explain each policy* converts a config step into a lesson.

---

## Prompt 2 — Wire the contact form to Supabase

**Goal:** stop the contact form from lying. Right now it says "message sent" and the data evaporates. After this prompt, every submission lands as a real row in your Supabase table.

**Before you paste:** open **Supabase → Project Settings → API** and copy two values into a sticky note:

- **Project URL** (looks like `https://abcd1234.supabase.co` )
- **anon public key** (a long string starting with `eyJ...` )

**Prompt — copy into Cursor Chat (Agent mode):**

You are pair-programming with a complete beginner. We are switching the contact form on `contact.html` from a "fake success message" to a "real database insert" using Supabase.

Please do the following, in order:

1. In `contact.html`, inside the `<head>`, add the Supabase JS client from the official CDN:  

```
<script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>
```
2. Create a NEW file called `supabase-config.js` in the project root with EXACTLY this content (we will paste real keys ourselves afterwards):  

```
const SUPABASE_URL = "PASTE_YOUR_URL_HERE";
const SUPABASE_ANON_KEY = "PASTE_YOUR_ANON_KEY_HERE";
const supabaseClient = supabase.createClient(SUPABASE_URL, SUPABASE_ANON_KEY);
```
3. In `contact.html`, link `supabase-config.js` BEFORE `script.js` (order matters, because `script.js` will use the `supabaseClient` variable created in `supabase-config.js`).
4. In `script.js`, update the EXISTING contact-form submit handler (do not duplicate it, do not delete the existing validation):
  - Read the four field values: `full_name`, `email`, `subject`, `message`.
  - Call: `await supabaseClient.from('form').insert([ { full_name, email, subject, message } ])`
  - If the response contains an error → keep the form visible and show a red error message: "Something went wrong. Please try again."
  - If the response is successful → hide the form and show the existing green success message, then reset the form fields.
  - `console.log` the full response object so we can debug if needed.
5. Add a plain-English comment above every new block of code so a beginner reading the file later can follow the logic.

Important guardrails:

- Do NOT remove or duplicate the existing submit listener – extend it.
- Do NOT change any element IDs or the success message text.
- Do NOT change the file names.

**After Cursor finishes:** open `supabase-config.js` and replace the two `PASTE_..._HERE` strings with your real URL and anon key. Save. Reload `contact.html` → submit a test message → switch to Supabase Table Editor → your row appears live. 🎉

### What this changes in your project:

- **HTML** ( `contact.html` ): one new `<script>` tag for the Supabase CDN in `<head>` ; one new `<script src="supabase-config.js">` placed **before** the existing `<script src="script.js">` .
- **CSS** ( `styles.css` ): nothing changes (the red error message can re-use existing styles).
- **JavaScript** ( `script.js` ): the existing submit handler is extended — it now reads form values, calls Supabase, and branches into success / error paths.
- **New file:** `supabase-config.js` holding your project URL + anon key.
- **Supabase (cloud):** every submission creates a new row in the `form` table.

### Why the prompt is written this way (prompt-engineering lens):

- **Explicit script order** ("BEFORE script.js") — this is the #1 invisible bug for beginners; calling it out in the prompt avoids it entirely.
- **Both success AND error paths** specified up-front — production-grade UX in one shot.
- **"Append, do not duplicate"** — a *scope-lock* phrase that stops the AI from rewriting your Part 1 code.
- **"Do not change element IDs"** — Part 2 prompts later depend on those IDs; we're future-proofing.

### Verify:

1. Submit a test message on `contact.html` — green success appears.
  2. Switch to Supabase → Table Editor → refresh → your row is there with `is_read = false` .
  3. Open browser DevTools → Console → you should see the response object logged.
-

# Prompt 3 — The Debug Template (use this every single time something breaks)

---

**Goal:** turn every red error in the browser console into a one-minute fix + a one-line lesson.

**Prompt — copy into Cursor Chat whenever you hit an error:**

```
You are pair-programming with a complete beginner. I am stuck.

I'm getting this exact error in [filename] when I try to [action]:

[paste the exact red text from the browser console here, nothing reworded]

Here is the relevant file (or feel free to read it directly):
[paste the file contents OR @-mention the file in Cursor]

Please:
1. Explain in ONE sentence what this error actually means, in plain English.
2. Identify the root cause.
3. Give me the precise fix — show ONLY the corrected lines, not the whole file.
4. Tell me ONE habit I could adopt to avoid this class of error in future.
```

**Why the prompt is written this way (prompt-engineering lens):**

-  **Persona reminder** — keeps the language beginner-friendly.
-  **Exact error text** — paraphrasing loses the keyword the AI needs to recognise the bug.
-  **"Corrected lines, not the whole file"** — surgical fix; less risk of accidental regressions.
-  **"One habit"** — turns every bug into a 30-second learning loop. Compounds over a career.

**How to find the exact error text:**

- Press `F12` (or right-click → **Inspect**) → **Console** tab.
  - The red lines are errors. Click the small triangle to expand. Copy the *first* red line — that's the original error; everything else is the trail.
-

## Prompt 4 — Build the Admin Dashboard

---

**Goal:** create a private page ( `admin.html` ) where you can see every contact-form message, mark it as read, and filter "unread only" — all reading live from your Supabase table.

**Prompt — copy into Cursor Chat (Agent mode):**

You are pair-programming with a complete beginner. We are adding an admin dashboard to the existing portfolio. Please reuse the same design system from the existing styles.css – no new fonts, no new colour scheme.

Create a NEW page admin.html in the project root.

NAV (re-use, do not redesign):

- Copy the EXACT navigation bar from index.html.
- Add a 5th link "Admin" pointing to admin.html.
- Mark the "Admin" link as active ONLY on admin.html (don't touch the other 4 pages' nav for now).
- Link styles.css, then supabase-config.js, then script.js – in that order (because script.js uses supabaseClient).

PAGE STRUCTURE:

1. Page header: h1 "Inbox" + small subtitle "All contact-form messages, newest first."
2. A toolbar row directly below the header:
  - LEFT: a total-message counter (e.g., "📬 12 messages").
  - RIGHT: a toggle labelled "Unread only" (use a styled checkbox).
3. A responsive grid of message cards:
  - CSS Grid, auto-fit, minmax(320px, 1fr), 20px gap.

EACH MESSAGE CARD:

- Top row: subject (h3, 18px) on the left; relative time on the right (e.g., "2 hours ago"). Use a small helper function timeAgo(date) for the relative time string.
- Sender line: "full\_name · email" (14px, muted colour).
- Body: the message text (15px). Clamp to a maximum of 4 lines using -webkit-line-clamp (with the standard overflow:hidden + display:-webkit-box setup). Show ellipsis if it overflows.
- Bottom-right: a "Mark as Read" button (small, primary style, matches existing button styles in styles.css).
- VISUAL STATES:
  - UNREAD card → 4px solid left border in soft purple (#a78bfa) + white background.
  - READ card → grey left border + soft grey background (#f3f4f6) + 60% opacity to feel "out of the way".

JAVASCRIPT (append to script.js – do NOT remove or alter the existing hamburger or contact-form code; only run the admin code if the inbox grid element actually exists on the current page):

1. On admin.html load, fetch all rows from the "form" table ordered by created\_at DESC, then render them as cards in the grid.
2. Update the total counter with the row count.
3. Wire the "Mark as Read" button on each card:
  - Run a Supabase UPDATE setting is\_read = true for that row's id.
  - On success, update ONLY that one card's class / styles (no full reload, no re-fetch).
4. Wire the "Unread only" toggle:
  - When checked, hide every card that already has the "is-read" class (use a CSS class on the grid, not a re-fetch).
5. Use a tiny helper function timeAgo(date) that returns "just now", "5

minutes ago", "2 hours ago", "3 days ago", "21 May 2026".  
6. Comment every block in plain English.

Important guardrails:

- Reuse colours, spacing and shadow tokens from the existing `:root` variables.
- Do not break any existing page or behaviour.

### What this changes in your project:

- **HTML** ( `admin.html` , new): a brand-new page with the same nav (plus an "Admin" link), a header, a toolbar, and an empty grid container that JavaScript fills.
- **CSS** ( `styles.css` ): a new `/* === ADMIN === */` section is appended with `.admin-grid` , `.admin-card` , `.admin-card.is-read` , the line-clamp rule, and the "Unread only" filter rule.
- **JavaScript** ( `script.js` ): a new block (gated on "does the admin grid exist on this page?") that fetches rows from Supabase, renders cards, handles "Mark as Read" updates, and toggles the unread filter. Plus a small `timeAgo()` helper.
- **Supabase**: every "Mark as Read" click sends an `UPDATE` query; the inbox load sends a `SELECT` .

### Why the prompt is written this way (prompt-engineering lens):

- ☒ **"Reuse the existing nav and design tokens"** — keeps the admin page visually consistent without making the AI redesign anything.
- ☒ **Two crystal-clear visual states (read / unread)** — descriptive UX, not vague "make it look like an inbox".
- ☒ **"Update only that one card"** — teaches *targeted DOM updates* instead of expensive full reloads.
- ☒ **"Only run the admin code if the grid element exists"** — defensive script.js that won't break the other 4 pages.

### Verify:

1. Open `contact.html` and submit 3 test messages.
  2. Open `admin.html` — all 3 appear, newest first, with the purple "unread" border.
  3. Click "Mark as Read" on one — the card greys out instantly (no reload). Refresh Supabase Table Editor — `is_read` is `true` for that row.
  4. Tick "Unread only" — the marked-as-read card disappears from view. Untick — it returns.
-

# Prompt 5 — Protect your secrets BEFORE the first `git push`

**Goal:** make sure your Supabase URL + anon key (and any future secret) never end up on GitHub.

**Prompt — copy into Cursor Chat:**

You are pair-programming with a complete beginner. We're about to push this project to a PUBLIC GitHub repository, so we need to protect secrets first.

Please:

1. Create a `.gitignore` file in the project root that ignores ALL of the following:

```
supabase-config.js # contains my API keys
.DS_Store # macOS clutter
Thumbs.db # Windows clutter
node_modules/ # if we ever install npm packages
.env # generic secrets file
.env.local # local-only secrets
```

2. Create a `supabase-config.example.js` file with the SAME shape as `supabase-config.js` but EMPTY string values, so any other developer (or future me on a new laptop) knows which two variables to define:

```
const SUPABASE_URL = "";
const SUPABASE_ANON_KEY = "";
const supabaseClient = supabase.createClient(SUPABASE_URL, SUPABASE_ANON_KEY);
```

3. Write a 2-sentence plain-English explanation, addressed to a beginner, of why we never commit API keys to GitHub — even when they are labelled "anon" or "public".

**What this changes in your project:**

-  **HTML / CSS / JS:** nothing in the website code.
-  **New files:** `.gitignore` (tells Git "skip these"), `supabase-config.example.js` (a safe template for other developers).
-  **Side-effect when you run `git push`:** `supabase-config.js` will stay on your laptop only — it never leaves.

**Why the prompt is written this way (prompt-engineering lens):**

-  **Safety first** — runs *before* `git init` so secrets never enter Git history at all.
-  **Both files in one prompt** — `.gitignore` AND `.example.js` are part of the same convention, so we ask for both together.
-  **Demand the "why" in plain English** — students remember the reason, not just the recipe.

**Verify:** the file tree now shows `.gitignore` and `supabase-config.example.js`. Open both — confirm they look correct.



## Push the project to GitHub (terminal commands — not a prompt)

First, on github.com, click **New repository** → name it `aanya-portfolio` → set it to **Public** → leave "Add a README" UNCHECKED → **Create repository**. Copy the HTTPS URL it shows you.

Open Cursor's built-in terminal ( `Ctrl + `` — the backtick key) and run, one at a time:

```
git init # 1. mark this folder as a Git project
git branch -M main # 2. rename the default branch to "main"
git add . # 3. stage every file (except those in
 .gitignore)
git commit -m "Initial portfolio + Supabase integration" # 4. snapshot it
git remote add origin https://github.com/<your-username>/aanya-portfolio.git
git push -u origin main # 5. send everything up
```

Refresh GitHub → your code is live in the repo. From now on the update loop is just `add → commit → push`.



## Deploy live with GitHub Pages (six clicks)

1. Your repo on GitHub → **Settings** tab.
2. Left sidebar → **Pages**.
3. **Build and deployment** → **Branch**: switch from `None` to `main`, folder `/ (root)` → **Save**.
4. Wait 30–60 seconds → refresh the page.
5. A green box appears with your live URL: `https://<your-username>.github.io/aanya-portfolio/`
6. Click it. **You are live on the internet**. Share it.

**Your update loop forever after:**

```
git add .
git commit -m "describe what you changed"
git push
wait ~30s, refresh the live URL
```

# Bonus — Rapid Prototyping: Timer App (10 minutes, deployed)

The portfolio took two sessions. This proves the *prompting* skill transfers to any small idea.

**Setup:** in Cursor → **File** → **New Window** → **Open Folder** → create a folder called `timer-app` on your Desktop → open it. (Brand-new project, clean slate.)

## Prompt 6 — Timer v1 (build it)

You are pair-programming with a complete beginner. I want a single-page web timer app.

Create 3 files in the current folder – `index.html`, `styles.css`, `script.js` – and link `styles.css` in `<head>` and `script.js` just before `</body>`.

Features:

- One big input field where the user types a number of SECONDS.
- Three buttons: Start, Pause, Reset.
- A big circular display in the centre showing the remaining time as MM:SS.
- When the timer reaches 0, play a short alarm sound from this royalty-free URL (use a hidden `<audio>` element):  
`https://actions.google.com/sounds/v1/alarms/alarm_clock.ogg`

Visual style:

- Dark background `#0f172a`, neon green digits `#34d399`, soft glow on the digits.
- Everything centred on the page; works on mobile (use flexbox + responsive font sizes with `clamp()`).

Quality bar:

- It must work properly with NO console errors when I open `index.html`.
- Comment every block of code in plain English so a beginner can read it.

**What this changes:** creates 3 brand-new files (`index.html`, `styles.css`, `script.js`) in the `timer-app` folder — none of your portfolio files are touched.

**Why this prompt:** persona + numbered features + exact resource URL + explicit quality bar ("no console errors") leaves zero ambiguity.

## Prompt 7 — Timer v2 (iterate without breaking)

Improvements to the existing timer (do NOT break existing behaviour):

1. Replace the alarm sound URL with this jingle instead:  
`https://actions.google.com/sounds/v1/alarms/digital_watch_alarm_long.ogg`
2. When the timer is under 10 seconds remaining, the circular display should gently pulse a red glow once per second (use a CSS @keyframes animation, only triggered by a JS class toggle).
3. On hover, all three buttons should lift 2px and gain a subtle drop shadow.
4. Add a small "Quick presets" row directly above the input field, with four pill-shaped buttons: 1m, 5m, 10m, 25m. Clicking a preset fills the input with the equivalent number of seconds (60, 300, 600, 1500).

**Why this prompt:** classic **anchor + delta** iteration — four changes, scope locked with "do not break existing behaviour".

**Deploy it:** repeat the GitHub + Pages steps. Two deployed apps in one day. 🚀



## You did it

You're now a **full-stack developer who ships in a single session**.

Pick one piece of homework:

1. Add a "Reply via Email" button on each admin card ( `mailto:` link pre-filled with the sender's email + subject prefixed with "Re:").
2. Add **Supabase Auth** so only *you* (when logged in) can see `admin.html`.
3. Buy a custom domain on Namecheap and point it at your GitHub Pages URL.
4. Build a brand-new tiny app entirely from prompts — and deploy it before you sleep.

# Part 2 — Training Guide (Trainer Edition)

---

## Learn Web Development with AI — From Static Site to Deployed Full-Stack App

**Duration:** ~3 hours (with Q&A breaks) **Prerequisite:** Part 1 completed. Students arrive with the `aanya-portfolio` folder intact. **Outcome:** The same portfolio is now visually stunning (Mega Prompt warm-up), backed by a real Supabase database, has an Admin Dashboard, lives on GitHub, and is **deployed live** via GitHub Pages.

---

### Audience reminder (read before you teach)

The class is **mixed-skill** — some students have never written code, some are technical. Every prompt in this Part is written so that:

- **Non-technical students** can copy, paste, and run with confidence — the prompts read like a clear brief, not a spec sheet.
- **Technical students** still get rigour, because every spec inside the prompts uses numbers, hex codes, and named patterns (no vague adjectives).

After each prompt, point students at:

1. The **"What this changes"** card (HTML / CSS / JS / new files), so they can mentally follow along even if they're not reading the code.
  2. The **"Why the prompt is written this way"** lens, which is the prompt-engineering lesson — that is the part that will compound for their careers.
- 

### The Bridge from Part 1

"Yesterday your contact form said *'message sent'* and lied. Today we make it tell the truth — and we put your whole site on the internet for real, for free, in three hours. And **before** we start, in a single prompt, we're going to give your portfolio a glow-up so dramatic you'll genuinely be excited to wire it up."

Recap on whiteboard (2 min):

- Part 1 = **Front-end** (skeleton, skin, brain — but no memory).
  - Part 2 = **Polish** (Mega Prompt) → **Back-end** (memory) → **Version control** (history) → **Deploy** (public address).
-

# How This Guide Is Structured

Each unit:

1. **Concept** — the idea, plain English.
2. **Prompt** — copy from `prompts.md`.
3. **Prompting Strategy** — *why* the prompt looks the way it does (this is the part the room should remember in 5 years).
4. **What this changes (HTML/CSS/JS callout)** — explicit, so non-technical students can follow along visually.
5. **Trainer Notes** — what to demo, what to watch for, what to say.

There are **five Q&A breakpoints**. Don't skip them.

---



# Module 0 — The Mega Prompt (warm-up • ~15 minutes)

**This is new. Run it first.** It does not introduce a new concept; it gives the room a *visceral* "wow" so the rest of Part 2 lands on a beautiful canvas instead of the bare-bones Part 1 look.

## 0.1 Concept

We will use *one* big consolidated prompt to upgrade the visual layer of every page in the Part 1 portfolio: glassmorphism nav, animated hero gradient blobs, dark/light theme toggle, scroll-reveal animations, gradient text, frosted cards, float-label inputs, a projects-page marquee, an animated stats strip on About, and a tiny page-load splash.

No new libraries. No build tools. Same 6 files. Runs by double-clicking the HTML.

## 0.2 The Prompt

See `mega-prompt.md` for the full text. Open it on a second monitor and ask a student to read out the **Goal** section while you prepare to demo.

## 0.3 Prompting Strategy (highlight to the room)

- **Consolidation taken to its limit** — what would have been 15 separate prompts is one carefully structured one. This is *the* power-move of senior prompting.
- **Numbers, hex codes, durations, easings** everywhere → multiple students running it independently will get *visually consistent* results.
- **Two-list structure: "DO" + "DO NOT"** — explicit guardrails (no new libraries, do not rename files, do not change copy) prevent the AI from "improving" things it shouldn't.
- **Demand deliverables** — the AI is asked to produce a short summary of files touched + 3 test instructions + the single most impactful change. This converts a build into a learning artefact.
- **Accessibility baked in** — `prefers-reduced-motion` rule is part of the prompt itself; nobody adds it later.

## 0.4 What this changes (cheat-sheet to project on screen while it runs)

- **HTML:** theme-toggle button added to nav across all 4 pages; `.reveal` classes sprinkled; marquee on Projects; stats strip on About. Content, links, IDs untouched.
- **CSS:** new design tokens in `:root` + `[data-theme="dark"]` override; glass nav; animated hero blobs + noise; gradient borders on hover; float-label inputs; marquee keyframes; reveal keyframes; reduced-motion guard.
- **JavaScript:** six modules appended on top of existing code — theme persistence, scroll progress bar, IntersectionObserver reveals, nav shrink-on-scroll, animated number counters, page-load splash. Hamburger + form handler from Part 1 left untouched.
- **New files:** none — same 6 files.

0.5 Trainer demo (live)

- 1. Open the folder in Cursor → Cursor Chat → Agent mode.
- 2. Open mega-prompt.md → copy the entire fenced prompt → paste → Enter.
- 3. While Cursor edits, narrate the highlights using the cheat-sheet above. Estimated edit time: 2–4 min.
- 4. When it finishes:
  - Reload index.html (double-click) → splash → hero blobs → scroll to skills → hover → notice border glow.
  - Click moon/sun in the nav → entire site flips. Refresh → theme remembered.
  - Open about.html → scroll to stats strip → counters tick up; timeline rows fade in.
  - Open projects.html → marquee scrolling; hover a card → image zooms, "View Case Study" slides up.
  - Open contact.html → focus an input → label floats; submit a test message → existing green success still works.
  - Resize the window to phone width → hamburger still functional, grids stack.

0.6 Bridge sentence

"That entire transformation was a single prompt. Now imagine running 5 of those in a real product. **That** is what AI + a clear brief unlocks. Now let's give this beautiful website a real memory."

Module 1 — Full-Stack in One Sentence

| ASPECT   | FRONT-END (CLIENT) | BACK-END (SERVER)            |
|----------|--------------------|------------------------------|
| Lives in | The browser        | A server / cloud             |
| Made of  | HTML, CSS, JS      | Database, server logic, APIs |
| Job      | Show & feel        | Remember & secure            |

A Full-Stack Developer can do both. By the end of today, **you** can do both.

Module 2 — Databases for Humans

2.1 The Excel-killer analogy

Ask the room: "Could we just store messages in an Excel file?" Let them argue. Then:

|                                     | EXCEL               | REAL DATABASE (SUPABASE) |
|-------------------------------------|---------------------|--------------------------|
| 1M rows                             | crashes laptop      | feels normal             |
| Two people editing                  | one wins, one loses | both work in parallel    |
| "Show me all unread from this week" | filters & formulas  | one SQL line             |

## 2.2 Table anatomy (sketch on whiteboard)

For a contact-form table called `form` :

| COLUMN                  | TYPE                                | WHY                             |
|-------------------------|-------------------------------------|---------------------------------|
| <code>id</code>         | bigint, auto-increment, primary key | Unique handle for every message |
| <code>created_at</code> | timestampz, default now()           | Sort & track                    |
| <code>full_name</code>  | text                                | User input                      |
| <code>email</code>      | text                                | User input                      |
| <code>subject</code>    | text                                | User input                      |
| <code>message</code>    | text                                | User input                      |
| <code>is_read</code>    | boolean, default false              | Admin workflow flag             |

**Trainer tip:** Show this table on slide. The next prompt will *generate the SQL for exactly this table*.

## Q&A Break #1 (5 min)

Ask: "What other data could you store in a similar table?" (Newsletter signups, event RSVPs, bug reports — all the same shape.)

# Module 3 — Generating SQL with AI

## 3.1 Concept

Don't memorise SQL. Describe the table in English, let the AI write the `CREATE TABLE` statement.

## 3.2 Prompt 1 — Generate SQL

See `prompts.md` → **Prompt 1**.

## 3.3 Prompting Strategy

-  **Persona for the AI** ("Postgres expert helping a complete beginner") → output is correct *and* explained simply.
-  **Numbered columns + exact types** → zero ambiguity.
-  **Demand explanation** → Rule #5. This is the moment a non-technical student understands "primary key" the way they understand "row number in Excel".

## 3.4 What this changes

- Nothing in the website yet.

- **Supabase only:** a new table named `form` appears in Table Editor with 7 columns.

### 3.5 Trainer demo — Supabase signup

1. Open **supabase.com** → Start your project → sign in with GitHub.
2. **New project** → name: `web-dev-with-ai` → strong DB password (save it!) → region: Mumbai/Singapore.
3. Wait ~60s for provisioning.
4. Left sidebar → **SQL Editor** → paste the AI-generated SQL → **Run**.
5. Sidebar → **Table Editor** → confirm `form` table exists with all 7 columns.

### 3.6 Optional row-level security

If the room has 10+ minutes of buffer, run **Prompt 1b** (RLS). Otherwise mention it as homework — Supabase auto-enables RLS, but for the classroom demo we'll temporarily disable it on the table so the form can write without auth.

---

## Module 4 — Connect Front-End to the Database

### 4.1 Concept

Two things every client app needs to reach Supabase:

- **Project URL** — the address.
- **Anon Key** — the public-facing API key.

Both are in **Supabase** → **Project Settings** → **API**. Copy them somewhere safe; we'll paste into a config file.

### 4.2 Prompt 2 — Wire the contact form to Supabase

See `prompts.md` → **Prompt 2**.

### 4.3 Prompting Strategy

-  **Explicit script ordering** ("BEFORE script.js") — the invisible-but-critical detail beginners always miss; calling it out in the prompt eliminates the bug entirely.
-  **Both error and success paths defined** — production-grade UX in one prompt.
-  **"Do not duplicate the handler"** — locks scope; protects Part 1 code.
-  **"Do not change element IDs"** — future-proofing for the admin dashboard prompt.

### 4.4 What this changes (point at this while Cursor edits)

-  **HTML ( `contact.html` ):** 2 new `<script>` lines — Supabase CDN in `<head>`, `supabase-config.js` placed **before** `script.js` at the bottom.
-  **CSS:** untouched.
-  **JS ( `script.js` ):** the existing submit handler is *extended* — it now reads values, calls `supabaseClient.from('form').insert([...])`, and branches into success / error.
-  **New file:** `supabase-config.js` (holds URL + anon key).

## 4.5 Live test

1. Paste actual URL + anon key into `supabase-config.js`.
2. Reload `contact.html` → submit a test message.
3. Switch to Supabase → Table Editor → refresh → **the row appears live**.

The room should audibly react. This is the moment the project becomes *real*.

---

## Module 5 — AI-Assisted Debugging (Live)

Expect at least one student to hit a CORS / "supabase is not defined" / wrong key error. Use it. **Don't fix it for them — show them the workflow.**

### 5.1 The four-step debug loop

1. **Identify** — F12 → Console tab → read the red text.
2. **Capture** — copy the exact error message.
3. **Prompt** — paste the **Prompt 3** template from `prompts.md`.
4. **Apply** — paste the fix, refresh, verify.

### 5.2 Prompting Strategy

-  **Show, don't tell** — paste the actual error verbatim; never paraphrase.
-  **"Corrected lines, not the whole file"** — minimises noise and accidental regressions.
-  **"One habit to avoid this in future"** — every bug becomes a 30-second lesson.

### 5.3 What this changes

Depends on the bug, of course. The common ones today:

- Wrong key in `supabase-config.js` → fix is one string.
  - Script-order mistake → fix is reordering two `<script>` tags in `contact.html`.
  - Disabled RLS missed → fix is a one-line policy in Supabase SQL editor.
- 

## Q&A Break #2 (10 min)

This is the *bigger* break. Walk the room — most students will be debugging here. Celebrate the first student whose row hits the database.

---

## Module 6 — The Admin Dashboard

### 6.1 Concept

We just *wrote* to the database. Now we *read* from it — and update it.

## 6.2 Prompt 4 — admin.html

See `prompts.md` → Prompt 4.

## 6.3 Prompting Strategy

-  "Reuse the existing nav and design tokens" — keeps the admin page consistent without making the AI redesign anything.
-  Two clear visual states (read / unread) — descriptive UX, not vague "make it look like an inbox".
-  "Re-render that one card, not full reload" — gentle introduction to *targeted DOM updates*.
-  "Only run the admin code if the grid element exists" — defensive JS that won't break the other 4 pages.  
Important pattern to call out: this is how a single `script.js` can serve a multi-page site.

## 6.4 What this changes

-  **HTML:** brand-new `admin.html` with nav (5 links now), header, toolbar, and an empty grid container.
-  **CSS:** new `/* === ADMIN === */` section in `styles.css` with `.admin-grid`, `.admin-card`, `.admin-card.is-read`, `line-clamp`, and the "Unread only" filter rule.
-  **JS:** new admin block (gated on grid existence) — fetch, render, mark-as-read, filter; plus `timeAgo()` helper.
-  **Supabase:** SELECT on load; UPDATE on Mark as Read click.

## 6.5 Trainer demo

1. From `contact.html`, submit 3–4 test messages.
2. Open `admin.html` → see them appear, newest first.
3. Click "Mark as Read" → watch the card visually fade.
4. Refresh Supabase Table Editor → `is_read` flipped to `true` for that row.
5. Tick "Unread only" → the read card disappears.

---

## Q&A Break #3 (5 min)

Ask: "What is the **one** other admin feature you'd want?" Collect 2–3 ideas. Pick the simplest and prompt for it live — pure improvisation; great confidence-builder.

Examples that will probably come up + a one-line prompt for each:

- **Search by sender** → "Add a search input in the admin toolbar that filters the visible cards by `full_name` or `email` (client-side, no Supabase call)."
  - **Delete message** → "Add a small X button on each card that runs `supabaseClient.from('form').delete().eq('id', row.id)`, confirms first, then removes the card from the DOM."
-

# Module 7 — Version Control with Git & GitHub

## 7.1 Concept — the moving-houses analogy

| STEP | REAL LIFE                                      | GIT COMMAND                      |
|------|------------------------------------------------|----------------------------------|
| 1    | Stuff scattered in your house                  | working directory                |
| 2    | Pile what you want to move in the drawing room | <code>git add .</code>           |
| 3    | Pack into labelled boxes                       | <code>git commit -m "msg"</code> |
| 4    | Load truck → new house (cloud)                 | <code>git push</code>            |

**Git** is the key. **GitHub** is the car. Git tracks; GitHub stores.

## 7.2 Setup (do once)

```
macOS
brew install git

Windows: download from git-scm.com/download/win
```

Then **once** per machine:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

## 7.3 Protect secrets first — Prompt 5

**Critical:** before anything else, create a `.gitignore` or your Supabase keys will be public on GitHub.

See `prompts.md` → **Prompt 5**.

## 7.4 Prompting Strategy

-  **Safety first** — the prompt runs *before* `git init`, so secrets never enter Git history at all.
-  **Both files in one prompt** — `.gitignore` + `.example.js` are part of the same convention; consolidate.
-  **Demand the "why" in plain English** — students will remember the *reason* long after they forget the syntax.

## 7.5 What this changes

-  **HTML / CSS / JS:** untouched.
-  **New files:** `.gitignore`, `supabase-config.example.js`.

## 7.6 The five terminal commands they will type

```
git init
git branch -M main
git add .
git commit -m "Initial portfolio + Supabase integration"
git remote add origin https://github.com/<your-username>/aanya-portfolio.git
git push -u origin main
```

## 7.7 Trainer demo — create the repo

1. GitHub → **New repository** → name `aanya-portfolio` → **Public** → do NOT initialise with README → Create.
2. Copy the HTTPS URL.
3. In Cursor's integrated terminal ( `Ctrl + `` ): run the five commands above.
4. Refresh GitHub → files appear (but **not** `supabase-config.js` — confirm this by checking the file tree; this is the moment `.gitignore` proves itself).

# Module 8 — Deploy with GitHub Pages

## 8.1 Step-by-step (project on screen, students follow)

1. Repo → **Settings** tab.
2. Left sidebar → **Pages**.
3. **Build and deployment** → **Branch**: change `None` → `main` · folder `/ (root)` → **Save**.
4. Wait 30–60 seconds → refresh → green box: `Your site is live at https://<username>.github.io/aanya-portfolio/`
5. Click it. **You are live on the internet.**

## 8.2 Smoke test

- Open the live URL on your phone (mobile responsive check + the Mega Prompt animations look great on touch).
- Submit a test message → check Supabase table.
- ⚠️ Note: `admin.html` will *not* work on the live URL until students paste their Supabase keys into a deployed config file. For the classroom we keep `admin.html` as a local-only page; productionising it is in the homework (Auth + RLS + env management).

## 8.3 Iteration loop forever after

```
git add .
git commit -m "describe the change"
git push
wait ~30s - site auto-updates
```

## ● Q&A Break #4 (10 min)

This is the **emotional peak** of the course — they have a live URL. Let it breathe. Encourage them to share the link on LinkedIn / WhatsApp. Walk around with high-fives.

## Module 9 — Rapid Prototyping Bonus: A Timer App

### 9.1 Why this module

The portfolio took two sessions. The next demo takes **ten minutes** — and proves the prompting skill is *generalisable*. Any small idea is now in reach.

### 9.2 Setup

File → New Window → Open Folder → create `timer-app` on Desktop → open in Cursor.

### 9.3 Prompt 6 — Timer v1

See `prompts.md` → **Prompt 6**.

### 9.4 Prompt 7 — Iterate

See `prompts.md` → **Prompt 7**.

### 9.5 Prompting Strategy

-  **Resource URLs provided** — no "find a sound effect" hand-waving.
-  **"It must work properly with no console errors"** — explicit quality bar.
-  **Iterative refinement in one prompt** — four enhancements, scope-locked with "do not break existing behaviour".

### 9.6 What this changes

-  Three brand-new files in `timer-app/`. None of the portfolio is touched.

## 9.7 Deploy in 60 seconds

Repeat the GitHub + Pages workflow. By the time they walk out of the room, students have **two** deployed apps.

---

# Module 10 — Wrap-up

## 10.1 What you can now do (read out loud)

-  Run a single mega-prompt that visually transforms an entire site.
-  Build a multi-page front-end from scratch via prompts.
-  Design a database table from plain English.
-  Connect a website to a cloud database (Supabase).
-  Build admin tooling that reads + updates real data.
-  Debug errors with AI as your pair.
-  Use Git to track changes and recover from mistakes.
-  Push code to GitHub and deploy live with GitHub Pages.
-  Spin up a brand-new prototype in ten minutes.

That is **full-stack web development**. Six months ago this was a multi-week curriculum.

## 10.2 Where to go next (signpost; don't teach)

- **Supabase Auth** — let users sign up and log in.
- **Row-Level Security** — proper production-grade rules.
- **Custom domain** — replace `username.github.io` with `aanyasharma.com`.
- **A framework** — when you outgrow plain HTML, look at React or Next.js.
- **Automation tools** — Make.com, n8n: trigger emails/Slack pings when a new row is inserted.

## 10.3 Homework (pick one)

1. Add a "Reply via Email" button on each admin card that opens a pre-filled `mailto:` link.
2. Add Supabase Auth so only *you* (logged in) can see `admin.html`.
3. Buy a domain on Namecheap, point it at GitHub Pages.
4. Build a *new* small app entirely from prompts — and deploy it.

---

## Q&A Break #5 — Final (15 min)

Common questions to anticipate:

- *"Is Supabase free forever?"* → Generous free tier; production projects upgrade only when traffic grows.
- *"Can I delete a record?"* → Yes; same pattern as update, just `.delete()`.
- *"What if I want to email myself when a message arrives?"* → Supabase Database Webhooks → Make.com / n8n. (Tease only.)

- *"Is my anon key dangerous in the repo?"* → It is *designed* to be public — but if you ever add a service-role key, that one **never** goes in Git. Always.
  - *"Can I use this for a client website?"* → Yes. Many freelancers ship exactly this stack today.
- 

## Appendix — The Five Rules, restated (still true)

1. **Numbers beat adjectives.**
2. **Persona before prompt.**
3. **One big prompt > five small ones.**
4. **Iterate with anchor + delta.**
5. **Always demand the "why".**

The technologies change every year. These five rules don't.  
Welcome to web development. Go build something.